

Analyse, modélisation, documentation et conception : Combat de robots

Objectifs - Concevoir et développer une application console client-serveur en utilisant la méthode de communication par socket synchrone

- Votre application devra permettre à 2 joueurs de s'affronter dans un combat de robots en réseau (**à tour de rôle**) ;
 - Un des joueurs agit comme serveur et comme joueur; le second joueur se connecte à celui-ci en tant que client ;
 - Vous devrez pouvoir jouer sur 2 ordinateurs différents* (*application serveur sur un, le client sur un autre*).
- *Pour des raisons de sécurité réseau, ceci n'est pas nécessairement possible!**
- Vous devrez utiliser les mécanismes appropriés afin d'effectuer les communications entre le client et le serveur ;
 - Vous devrez permettre la saisie des informations afin d'établir correctement la connexion entre le client et le serveur (*adresse IP et port du serveur pour la connexion → **pas localhost***).

Date de remise (Projet final)

- À remettre au plus tard **Vendredi le 9 Octobre à 7h59**

Remise

- L'ensemble des documents de conception et d'analyse (*ceux demandés*)
- Remettre sur LÉA un document Word contenant :
 - Lien GIT de votre dépôt Git
 - Les noms des coéquipiers
 - La liste (*titre*) des documents produits
 - **Le document des changements apportés pour l'arrimage de vos projets**
 - Un petit post mortem des **éléments appris ou réalisés** dans le cadre du présent travail

Directives générales

- Ce travail doit être réalisé en équipe (*Équipes définies par l'enseignant*)
 - *Utilisez les mêmes équipes que celle du travail précédent*
- La solution sera compilée sur un environnement standard VS 2026 Community
 - **Une solution qui ne compile pas = programme non fonctionnel = ZÉRO**

Critères d'évaluation (Sommaire)

- Fonctionnement du programme et qualité de la documentation (3)
- Utilisation adéquate des concepts de la POO, validations, gestion d'exceptions et tests (1)
- Modularité, structure et architecture du code (1)

Consignes / Contraintes

En plus des contraintes et directives du projet initial voici les ajouts au projet :

Cette section définit les contraintes techniques, architecturales et fonctionnelles supplémentaires à intégrer dans le cadre du Mini-Projet 2 (*Combat de Robots*). Ces exigences visent à consolider la robustesse réseau, garantir la cohérence des échanges client-serveur et enrichir l'expérience de jeu.

Fiche de spécifications des exigences et contraintes supplémentaires

Liste des exigences et contraintes imposées

- Matrice de compatibilité Client/Serveur (*Tests d'interopérabilité*) → **voir l'exemple de grille**
 - **Exigence** : Tester systématiquement la compatibilité croisée entre toutes les versions ou déclinaisons de clients et de serveurs.
 - **Livrable** : Fournir une matrice de confirmation certifiant la réalisation effective de l'ensemble des tests de compatibilité pour toutes les permutations (A-B, A-C, A-D, B-C, B-D, C-D), s'appliquant aussi bien aux rôles client qu'aux rôles serveur.
- Gestion de l'aspect aléatoire du côté serveur (*Attaques et Blocages*)
 - **Exigence** : Introduire une dimension aléatoire (*RNG*) lors du traitement et de la résolution des attaques et des blocages.
 - **Architecture** : L'algorithme de génération aléatoire doit impérativement être implémenté côté serveur afin d'assurer l'intégrité de l'état du jeu et d'empêcher toute tentative de triche ou de désynchronisation côté client.
- Nouvelle fonctionnalité de jeu
 - **Exigence** : Concevoir et implémenter une nouvelle fonctionnalité originale de votre choix dans le jeu.
 - **Objectif** : Enrichir la boucle de gameplay (*ex. système de compétences à rechargement, bonus temporaires accumulables, principe de fuite, jauge d'endurance ou esquive*) pour apporter de la profondeur stratégique et accroître l'intérêt ludique.
- Ajustement et équilibrage des paramètres de jeu
 - **Exigence** : Ajuster finement les paramètres globaux régissant la dynamique de combat.
 - **Objectif** : Équilibrer les facteurs clés (*points de vie, délais de rechargement/cooldowns, puissance des attaques, fenêtres de parade*) afin de rendre la dynamique de partie fluide, nerveuse et équitable. (*Vous pouvez également modifier la dynamique des échanges si cela est pertinent et accepté par tous les membres de l'équipe*)

Modèle de matrice de compatibilité (*Exemple*)

Ci-dessous, un modèle de matrice synthétique à compléter et inclure dans le livrable afin de valider la compatibilité des différentes permutations :

Permutation Client-Serveur	Déclinaison Client / Serveur	Statut du Test	Remarques / Validation
A - B	Client A / Serveur B	Conforme / Validé	Test réseau OK
A - C	Client A / Serveur C	Conforme / Validé	Test réseau OK
A - D	Client A / Serveur D	Conforme / Validé	Test réseau OK
B - C	Client B / Serveur C	Conforme / Validé	Test réseau OK
B - D	Client B / Serveur D	Conforme / Validé	Test réseau OK
C - D	Client C / Serveur D	Conforme / Validé	Test réseau OK

→ **Faire également même logique mais inverser client/serveur**